Instituto Superior Técnico

Universidade de Lisboa

2º Semester 2025/2026

Data Analytics for Smart Grids

Regression over Autoregressive Forecast

Gustavo Curado, 106672

Rui Costa, 106197

Week 4 - March 27, 2026

Main Work

The objective of this work is to model and forecast a time series variable using regression-based approaches, while accounting for possible autocorrelation in the residuals. In particular, we aim to evaluate whether a simple Ordinary Least Squares (OLS) regression is sufficient, or whether more advanced techniques that explicitly address serial correlation provide better results.

Autocorrelation is a common issue in time series data, where residuals are not independent but instead depend on past values. This violates the classical assumptions of OLS and can lead to inefficient and biased inference. Therefore, several alternative methods are considered and compared.

To study this effect of autocorrelation, four different methods will be used: Ordinary Least Squares, Concrane Orcutt, Auto-regression and Auto-regression + load data.

Below are the training equations used for each model, as well as the methods used to solve them respectively:

- **OLS:**

$$i_{12t} = \beta_0 + \beta_1 i_{1wt} + \varepsilon_t \tag{1}$$

- **Cochrane–Orcutt (CO):**

$$i_{12t} - \rho i_{12t-1} = \beta_0(1-\rho) + \beta_1(i_{1wt} - \rho i_{1wt-1}) + u_t \tag{2}$$

  Solved iteratively by:

  - Estimating residuals and computing ρ:

$$\rho = (r_{t-1}^T r_{t-1})^{-1} r_{t-1}^T r_t \tag{3}$$

  - Transforming variables:

$$y_t^* = y_t - \rho y_{t-1}, \quad x_t^* = x_t - \rho x_{t-1} \tag{4}$$

  - Applying OLS to transformed data:

$$\boldsymbol{\beta} = (X^{*T} X^*)^{-1} X^{*T} Y^* \tag{5}$$

- **AR(1):**

$$i_{12t} = \beta_0 + \beta_1 i_{1wt} + \beta_2 i_{12t-1} + \varepsilon_t \tag{6}$$

- **AR(1) + Loads:**

$$i_{12t} = \beta_0 + \beta_1 i_{1wt} + \beta_2 i_{12t-1} + \beta_3 Load_t + \varepsilon_t \tag{7}$$

	β_0	β_1	β_2	β_3
OLS	0.201	0.092	-	-
CO	-0.117	0.179	-	-
AR	0.095	0.064	0.505	-
AR + Loads	-0.046	0.354	0.051	-0.112

Table 1: Values of the different fitting parameters for each algorithm used.

Additionally, he Durbin–Watson statistic was computed for each model. The statistic is defined such that values close to 2 indicate no autocorrelation, values below 2 indicate positive autocorrelation, and values above 2 indicate negative autocorrelation. An estimate of the autocorrelation coefficient ρ was also obtained. These two quantities are given by:

- **Durbin–Watson Statistic:**

$$DW = \frac{\sum_{t=2}^{T}(\varepsilon_t - \varepsilon_{t-1})^2}{\sum_{t=1}^{T}\varepsilon_t^2} \tag{8}$$

- **Autocorrelation Coefficient (ρ):**

$$\rho \approx 1 - \frac{DW}{2} \tag{9}$$

	Durdin Wattson	ρ
OLS	0.76125	0.619
CO	0.03573	0.982
AR	1.44507	0.277
AR+Loads	1.74218	0.129

As we can see, the values for the Durdin Wattson approach 2 as we move from OLS to AR to AR+Loads.This is expected, since progressively more information about the temporal structure of the data is being incorporated into the model. In particular, the inclusion of lagged terms allows the model to capture part of the serial dependence that was previously left in the residuals, thereby reducing autocorrelation and improving the validity of the regression assumptions. The outlier here is Cochrane-Orcutt, for which ρ is near one. This is most likely due to the fact that Cochrane-Orcutt only accounts for a lag of 1, and our variables likely have a lag higher than that.

The fitted values of β were then used on new data to see how well each of the models can forecast new data. These results, as well as their residuals, are plotted in Figures 1 and 2 respectively.

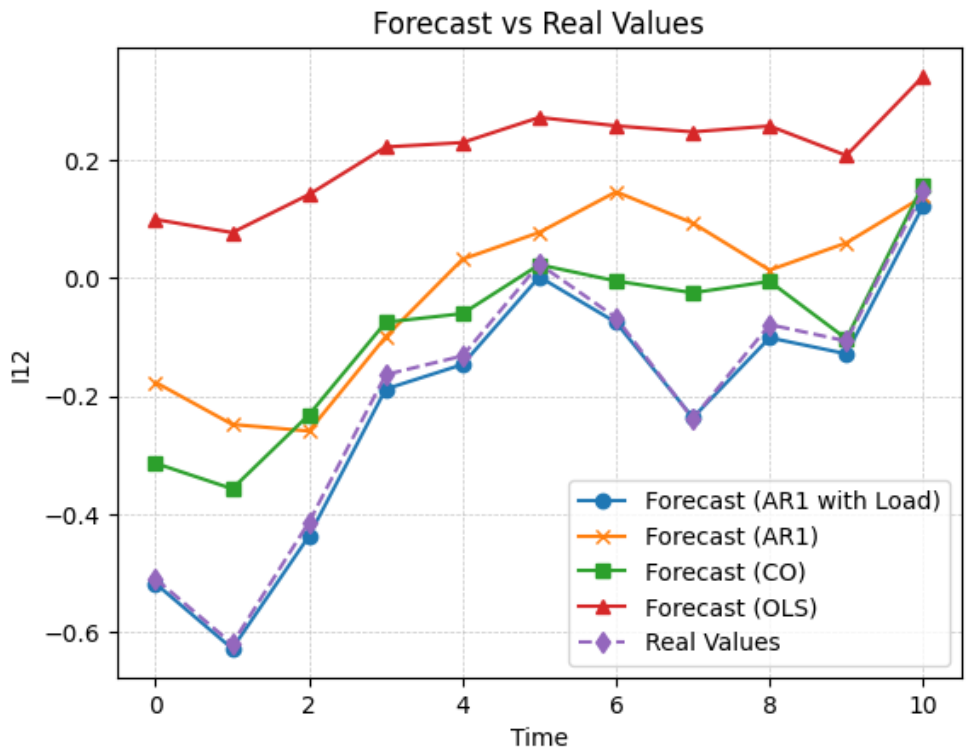

Figure 1: Forecast current for different estimation methods, compared to real values.

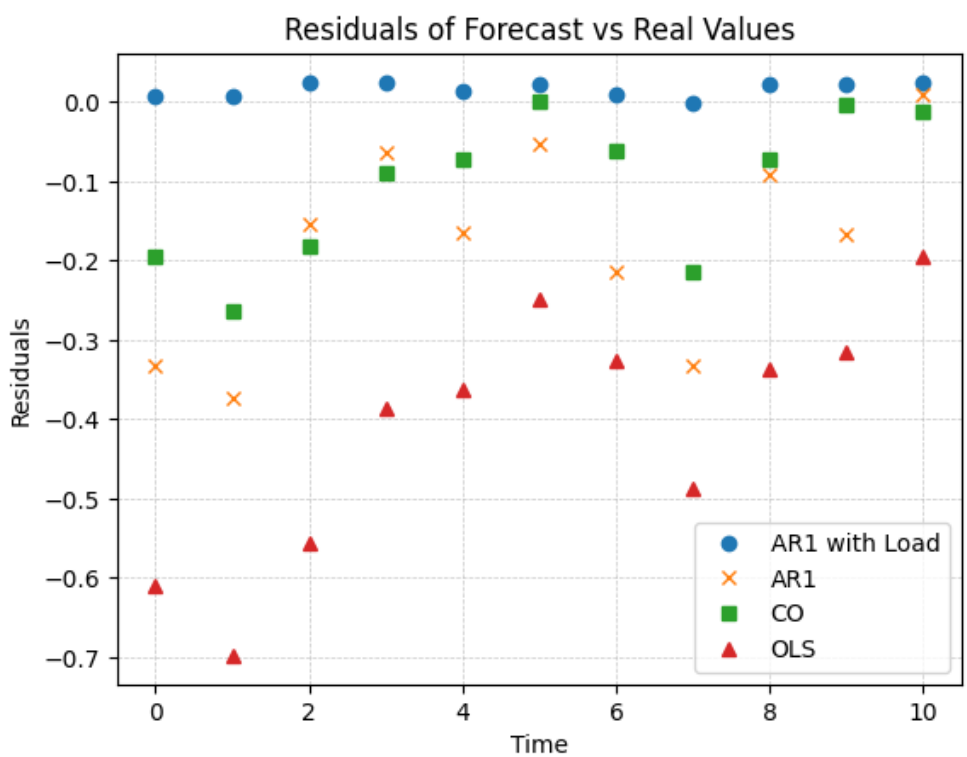

Figure 2: Residuals for different estimation methods.

These results show that, even though it lead to the highest value of ρ, the forecast done with CO

outperformed both OLS and AR. This suggests that correcting the error structure had a stronger impact on prediction accuracy than explicitly modeling the lagged relationships in those cases.

Nevertheless, AR + Loads is still the clear outperformer in this scenario, as it combines both temporal information and additional explanatory variables, leading to more accurate and stable forecasts.

Extra Challenge: Neural network

As part of the extra challenges, we explored the possibility of using a neural network to solve the forecasting problem. We want to evaluate if a more complex machine learning approach could improve upon the predictions made by the previous models.

The following code refers to the exact block that creates the network we are using. We initializes a multi-layer perceptron neural network for regression tasks, configured using five key parameters:

- `hidden_layer_sizes`: Establishes the network's internal architecture by creating a single hidden layer.
- `activation`: Applies the rectified linear unit function, allowing the model to learn non-linear relationships and efficiently handle gradient updates.
- `solver`: Dictates the optimization algorithm. This quasi-Newton optimizer is recommended for its fast convergence and strong performance on smaller datasets.
- `max`: Provides the solver with a generous allowance of up to 1,000 training epochs to ensure the model reaches convergence.
- `random`: Sets a fixed seed for the initial weight generation, ensuring that the model's results remain perfectly reproducible.

```
nn_model = MLPRegressor(
    hidden_layer_sizes=(10,), # Neuronios
    activation='relu',
    solver='lbfgs',           # Conjunto de dados pequenos - lbfgs;
    max_iter=1000,
    random_state=42
)
```

For this example, we chose the LBFGS solver over more common alternatives like Adam or SGD. Since our dataset is quite small, LBFGS is the more efficient choice, as it is designed to handle low data volumes with high precision.

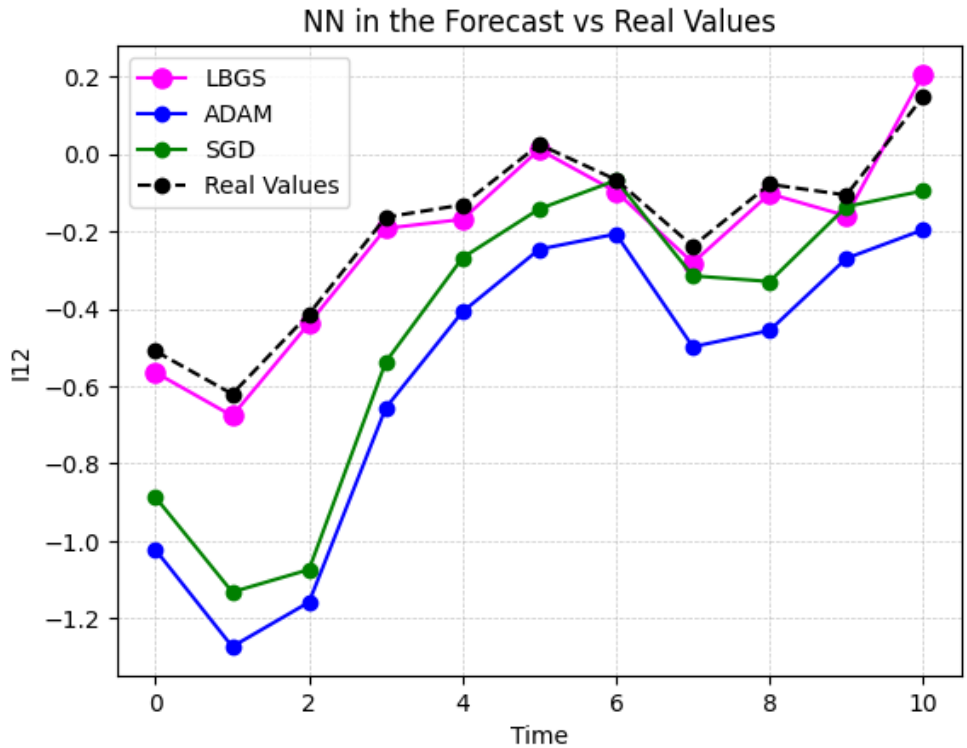

Figure 3: Comparison between the three possible solvers and the real values

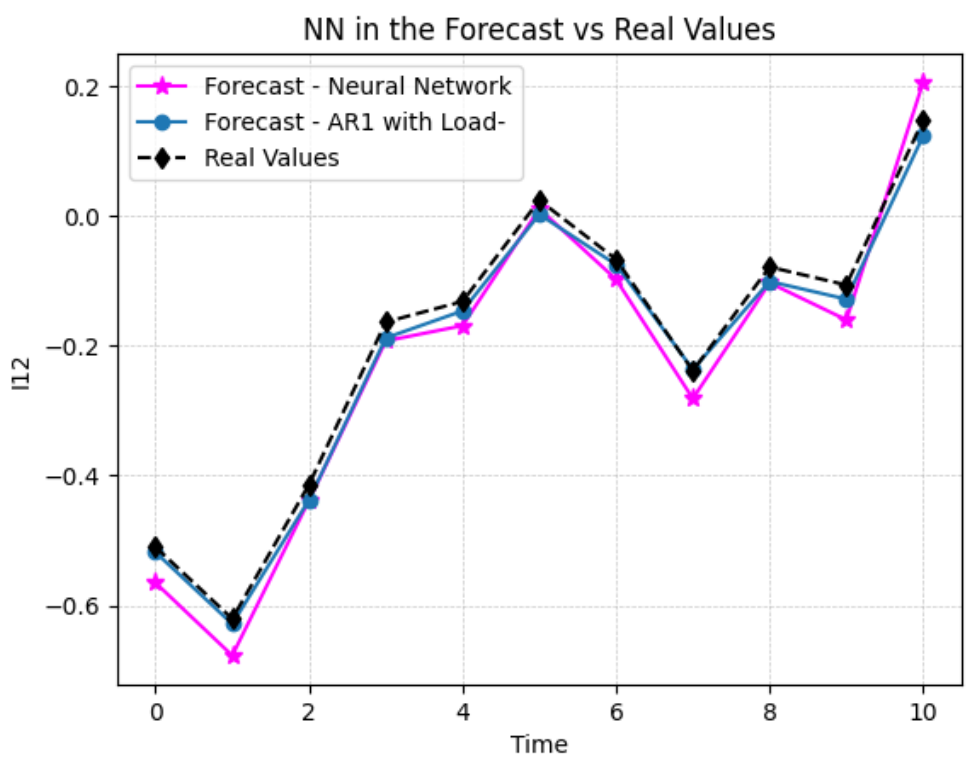

Figure 4: The two forecasts vs Real Values

Figure 4 compares the values of 'I12' against the predictions from two different models over a 10-step

count. While both the LBFGS trained neural network and the AR+Loads model capture the overall trend, the AR+Loads shows a tighter fit to the data. The neural network follows the general curve but presents larger gaps at specific intervals, indicating that the AR+Loads model was more effective for this particular dataset and configuration.

Error Calculation

The table below evaluates the performance of the neural network using different solvers by measuring the discrepancy between the model's forecasts and the real values. The mean squared error calculates the average of the squared differences between the predictions and the real values. The mean absolute error represents the average absolute magnitude of these differences. Both metrics are expressed as decimals in the original absolute units of the target variable, rather than as percentages. Consequently, values closer to zero indicate a smaller error margin, demonstrating that the model's predictions are closer to the real values. In this case, the LBFGS solver achieved the lowest error.

Solver	**MSE**	**MAE**
LBFGS	0.0017	0.0387
ADAM	0.1828	0.3850
SGD	0.1050	0.2572

Table 2: Comparison of forecast errors between LBFGS, ADAM and SGD solvers.

The trained neural network captures the data's general trend with a certain precision, yet the results reinforce a key forecasting principle: complexity doesn't always beat simplicity. Both the error metrics calculated and visual alignment show that the AR+Loads model outperforms the neural network. This suggests the data dynamics are likely linear, allowing the simpler model to deliver higher accuracy without the computational cost of deep learning.